

T2: Baseline Maintainability Benchmark Report

Team Beta — Sprint 1, Day 2 — 21 May 2026

Abstract

This report presents the complete Sprint 1 static-analysis baseline for Team Beta's iDaVIE codebase as required by deliverable T2 of the CS4443 assessment specification. **Four tool sets** are now incorporated: SonarQube Cloud for duplication, reliability and maintainability issues across 13,153 LOC (10,845 C# + 2,308 C++); Understand for the mandatory Chidamber and Kemerer (CK) object-oriented metric suite across 22 classes; NDepend v2026.1.5 for architecture violation detection, dependency cycle analysis and technical debt quantification; and CodeScene for hotspot identification, temporal coupling analysis and knowledge distribution. **This v5 amendment adds the DV8 substitution rationale. DV8 could not produce a valid DSM for the Unity scripting assembly — DSM coverage is fully satisfied by NDepend (spec Section 7.2 permits DV8 or NDepend). All five mandated tools now have documented outputs or justified substitutions.**

Four god-class candidates are confirmed by convergent evidence across all tools — **VolumeDataSet** (WMC 278, CBO 27, LCOM 94%, CodeScene health 1.92 and declining), **VolumeDataSetRenderer** (WMC 142, CBO 44, LCOM 96%, health 5.90), **VolumeInputController** (WMC 260, CBO 40, LCOM 95%, health 5.78 and declining), and **CanvassDesktop** (WMC 247, CBO 47, LCOM 96%, health 3.53). NDepend confirms **two mandatory architecture violations: a 7-namespace dependency cycle (ND1401) and 119-member mutual namespace dependency (ND1400)** — both must reach zero. CodeScene temporal coupling reveals that VolumeDataSet and VolumeDataSetRenderer have changed together in 103 revisions (48% coupling), confirming the architectural boundary between them is already broken in practice. Total technical debt is 31 days (NDepend rating C). All five ISO/IEC 25010 maintainability sub-characteristics remain high to critical risk.

Executive Summary

This report establishes the Sprint 1 static-analysis baseline for Team Beta's iDaVIE fork in accordance with deliverable T2. v5 adds the DV8 substitution rationale, completing all five mandated tool outputs or justified substitutions. Data sources: **SonarQube Cloud** (main, 20 May 2026); **Understand** CSV export (20 May 2026); **NDepend v2026.1.5** (20 May 2026, 115 source files); **CodeScene** (analysis 19 May 2026, 620 files, full git history 2018–2026).

Key findings across all four tools:

- **Four god-classes confirmed:** VolumeDataSet, VolumeDataSetRenderer, VolumeInputController, and CanvassDesktop — all breaching CK targets critically and all appearing as unhealthy or problematic hotspots in CodeScene.
- **Mandatory architecture violations (NDepend):** 7-namespace dependency cycle (ND1401) and CatalogData ↔ VolumeData mutual dependency (ND1400, 119 members). Spec requires 0 cycles — these are blocking issues for Sprint 2.
- **Strongest temporal coupling (CodeScene):** VolumeDataSet ↔ VolumeDataSetRenderer at 48%, 103 joint revisions — highest revision count in the codebase. The architectural boundary between these classes is already broken in practice.
- **New risk identified:** DesktopPaintController.cs (1,207 LOC, health 3.65, declining) — not in original CK baseline but confirmed by CodeScene as an actively deteriorating hotspot. ST6 ownership.
- **Technical debt:** 31 days total (NDepend). Debt rating C; 5d 2h effort to reach rating B. 12% of estimated development effort — within the 20% warning threshold.
- **Knowledge:** Code Familiarity 100% (all code written by currently active developers). Knowledge Islands 15% — rated Good. VolumeData directory has the highest coordination need.

1. Scan Provenance and Tooling

Parameter	Value
SonarQube — tool	SonarQube Cloud
SonarQube — project key	EPIC-Team-Beta_iDaVIE
SonarQube — branch / date	main · 20 May 2026
SonarQube — scanner	dotnet-sonarscanner via GitHub Actions
SonarQube — C# scope	Assets/Scripts/**/* .cs (7 sub-directories, 10,845 LOC)
SonarQube — C++ scope	native_plugins_cmake/**/* .cpp, *.cc, *.h (8 files, 2,308 LOC)
SonarQube — excluded	**/*.meta, **/Library/**, **/Temp/**, **/Obj/**, **/Build/**
Understand — tool	Understand (Scientific Toolworks) — CSV metrics export
Understand — date	20 May 2026
Understand — scope	All C# classes in Assets/Scripts/ (22 classes measured)
Understand — CK mapping	WMC=SumCyclomatic · CBO=CountClassCoupled · DIT=MaxInheritanceTree · NOC=CountClassDerived · RFC=CountDeclClassMethod · LCOM=PercentLackOfCohesion (Henderson-Sellers, %)
NDepend — tool	NDepend v2026.1.5
NDepend — date	20 May 2026 22:31
NDepend — scope	Assembly-CSharp.dll + Assembly-CSharp-Editor.dll (115 source files)
NDepend — runtime	.NET Framework 4.8.9325.0
CodeScene — tool	CodeScene Cloud
CodeScene — analysis date	19 May 2026 (report generated 21 May 2026)
CodeScene — scope	23,116 LOC across 620 files, 109 shown in knowledge map
CodeScene — git history window	Full history 2018 – May 2026
Coverage	0.0% — no test project wired to CI at Day 2 (expected)
DV8	SUBSTITUTED by NDepend DSM — DV8 could not produce a valid DSM output for the Unity scripting assembly (Assembly-CSharp.dll). DSM coverage is satisfied by NDepend dependency matrix and namespace coupling analysis (ND1400/ND1401), which fulfils spec Section 7.2 requirement for package/component dependencies as DSM. Spec Section 7.3 notes 'DV8 or NDepend' for DSM output.

2. Full Scan — Aggregate Summary (SonarQube)

Directory / Scope	LOC	Security	Reliability	Maintainability	Sec. Hotspots	Coverage	Duplications
Assets/Scripts (total C#)	10,845	0	40	783	4	0.0%	3.8%
FeatureData/	2,318	0	10	138	0	0.0%	0.6%
Menu/	699	0	5	30	0	0.0%	14.4% △
PluginInterface/	972	0	4	278	2	0.0%	7.7% △
Tools/	111	0	1	9	0	0.0%	0.0%
UI/	1,540	0	11	89	0	0.0%	5.7% △
VoiceCommands/	219	0	4	15	0	0.0%	0.0%
VolumeData/	4,986	0	5	224	2	0.0%	2.4%
native_plugins_cmake/ (C++)	2,308	0	3	315	0	0.0%	3.8%
TOTAL SCANNED	13,153	0	43	1,098	4	0.0%	3.8%

Source: SonarQube Cloud — EPIC-Team-Beta/iDaVIE, main, 20 May 2026. **Menu/** has the highest duplication at 14.4%, driven by PaintMenuController (20.4%) and QuickMenuController (14.9%). **PluginInterface/** has 278 maintainability issues and 7.7% duplication. No security issues detected in either C# or C++ layers.

3. Chidamber and Kemerer (CK) Metric Baseline (Understand)

The following table presents the mandatory CK suite per spec Section 7.1 for all 22 measured classes. Colour coding: **red** = breaches target, **amber** = exceeds 70% of limit, **green** = within target. WMC and CBO limits are split by class role: domain $\leq 20/\leq 14$; orchestrator/adaptor $\leq 40/\leq 25$. LCOM is Henderson-Sellers variant expressed as % (target $\leq 50\%$). DIT=5 for all MonoBehaviour subclasses reflects the Unity inheritance chain and is not a design defect.

Class / Sub-team	WMC $\leq 20/40$	DIT ≤ 4	NOC ≤ 5	CBO $\leq 14/25$	RFC ≤ 50	LCOM% ≤ 50	LOC	Assessment
VolumeDataSet (ST3)	278	1	0	27	4	94%	1535	CRITICAL
Config (ST1)	7	1	0	17	4	99%	136	LCOM ⚠
BenchmarkManager (ST1)	15	5	0	10	0	53%	101	LCOM ⚠
Delegates.cs (ST1)	—	—	—	—	—	—	6	Contract
FitsReader (ST2)	55	1	0	2	13	92%	589	Breach
AstTool (ST2)	0	1	0	0	0	0%	47	Clean
DataAnalysis (ST2)	14	1	0	5	4	88%	187	LCOM ⚠
NativePluginLoader (ST2)	20	5	0	6	1	64%	97	LCOM ⚠
VolumeDataSetRenderer (ST3)	142	5	0	44	0	96%	1089	CRITICAL
MomentMapRenderer (ST3)	27	5	0	32	0	84%	295	CRITICAL
VolumeInputController (ST4)	260	5	0	40	5	95%	1210	CRITICAL
VolumeCommandController (ST4)	123	5	0	22	0	88%	579	CRITICAL
QuickMenuController (ST4)	62	5	0	14	0	92%	331	Breach
PaintMenuController (ST4)	51	5	0	12	0	92%	258	Breach
TabsManager (ST4/ST6)	10	5	0	5	0	47%	67	Within
FeatureSetRenderer (ST5)	90	5	0	26	2	91%	497	CRITICAL
FeatureSetManager (ST5)	60	5	0	15	1	94%	360	Breach
FeatureMenuController (ST5)	89	5	0	16	0	79%	325	Breach
FeatureMenuCell (ST5)	60	5	0	15	0	86%	220	Breach
FeatureTable (ST5)	16	1	0	7	3	33%	112	Within
VoTable (ST5)	40	1	0	3	1	86%	199	Breach
CanvassDesktop (ST6)	247	5	0	47	0	96%	1525	CRITICAL

Notes: (1) AstTool metrics are all zero — the C# class contains only externDllImport declarations, implementation in ast_tool.cpp. (2) RFC shows 0 for most MonoBehaviour subclasses because Understand cannot resolve inherited Unity methods through the stub project; treat RFC=0 at DIT=5 as an underestimate. (3) Delegates.cs contains only delegate type declarations — no class body to measure.

4. Per-Sub-Team Baseline Summary

Full per-sub-team breakdowns as documented in v3 are unchanged. This section records the updated assessment status incorporating NDepend and CodeScene confirmation.

ST1 — VolumeDataSet.cs: CONFIRMED CRITICAL across all four tools. WMC 278, CBO 27, LCOM 94%, CodeScene health 1.92 (Unhealthy, declining). Highest-priority refactoring target in the codebase.

ST3 — VolumeDataSetRenderer.cs: CONFIRMED CRITICAL. WMC 142, CBO 44, LCOM 96%, CodeScene health 5.90. 48% temporal coupling with VolumeDataSet over 103 revisions — boundary already broken in practice.

ST4 — VolumeInputController.cs: CONFIRMED CRITICAL. WMC 260, CBO 40, LCOM 95%, CodeScene health 5.78 (declining). 38% temporal coupling with VolumeCommandController over 67 revisions.

ST6 — CanvassDesktop.cs: CONFIRMED CRITICAL. WMC 247, CBO 47, LCOM 96%, CodeScene health 3.53. CBO 47 is highest in codebase. CanvassDesktop ↔ VolumeDataSetRenderer: 21% coupling, 95 revisions. Additionally: DesktopPaintController.cs (health 3.65, declining) is a newly confirmed hotspot not previously in CK baseline — ST6 owns both files.

ST5 — FeatureSetRenderer / FeatureSetManager: HIDDEN COUPLING CONFIRMED by CodeScene: 60% temporal coupling, 38 revisions — these two classes change together more often than apart, indicating a missing interface boundary.

ST2 — FitsReader.cs / data_analysis_tool.cpp: data_analysis_tool.cpp scores 2.10 (Unhealthy) in CodeScene. FitsReader.cs: 67% coupling with data_analysis_tool.cpp (103 revisions) — expected wrapper/implementation relationship but warrants monitoring.

VideoMaker — VideoCameraController.cs: NDepend critical issue: VideoCameraController.Export() rated Critical severity. SaveFrames() creates raw threads (ND2500 violation). Must be triaged before production. CodeScene health 9.24 — structurally healthy but the threading issue is a functional risk.

5. NDepend Analysis Results

NDepend v2026.1.5 analysed Assembly-CSharp.dll and Assembly-CSharp-Editor.dll on 20 May 2026, covering 115 source files. Analysis duration: 13.2 seconds.

5.1 Quality Gates

Quality Gate	Actual Value	Threshold	Result
Critical Rules Violated	8 rules	0 rules	FAIL
Critical Issues	1 issue	0 issues	WARN
Blocker Issues	0	0	PASS
New Blocker/Critical/High Issues	0	0	PASS
Treat Compiler Warnings as Error	0	0	PASS
Percentage Debt	12%	≤30% fail / ≤20% warn	PASS
New Debt since Baseline	0	0	PASS
Debt Rating per Namespace	A/B range	—	PASS
Annual Cost Increase	0	2 man-days fail	PASS
Percentage Coverage	N/A	≥70%	SKIPPED (no test data)
Percentage Coverage on New Code	N/A	≥70%	SKIPPED (no test data)
Percentage Coverage on Refactored Code	N/A	≥70%	SKIPPED (no test data)

BUILD FAILS: 1 quality gate fails: Critical Rules Violated = 8 rules (threshold 0). The build must not be released until this is resolved.

5.2 Critical Rules Violated (8 of 8)

Category	Rule ID	Description	Severity
Code Smells	ND1003	Avoid methods too big, too complex	CRITICAL
Code Smells	ND1000	Avoid types too big	CRITICAL
Code Smells	ND1004	Avoid methods with too many parameters	CRITICAL
Architecture	ND1400	Avoid namespaces mutually dependent — 119 members across CatalogData ↔ VolumeData	CRITICAL
OO Design	ND1901	Avoid non-readonly static fields (VolumeDataSetRenderer.ThresholdMax)	CRITICAL
Naming	ND2012	Avoid having different types with the same name (CatalogDataSetRenderer)	CRITICAL
Naming	ND2005	Attribute class name should be suffixed with 'Attribute'	CRITICAL
Threading	ND2500	Don't create threads explicitly (VideoCameraController.SaveFrames())	CRITICAL

5.3 Architecture Violations — Dependency Cycles

MANDATORY VIOLATION: The spec requires 0 dependency cycles. NDepend has confirmed both ND1400 (mutually dependent namespaces) and ND1401 (namespace dependency cycles). These are blocking issues and must be resolved in Sprint 2.

ND1400 — Avoid Namespaces Mutually Dependent: 119 types and methods across CatalogData ↔ anonymous namespace and VolumeData ↔ anonymous namespace are mutually dependent. This is a circular architecture violation. The low-level namespace references the high-level namespace — a direct layering breach. Estimated debt: 30+ minutes to resolve the first pair; full resolution will require interface extraction.

ND1401 — Avoid Namespaces Dependency Cycles: A 7-namespace cycle has been confirmed involving VolumeData, UI, and VoTableReader among others. NDepend estimates 2 hours debt to break this cycle. Fixing the ND1400 mutual dependencies will resolve most of this cycle; the remainder requires 3+ namespace restructuring.

Assembly-level cycles: None detected. The assembly reference graph is cycle-free — the violations are at namespace level within Assembly-CSharp.

5.4 Assembly Metrics

Assembly	LOC	IL Instructions	Types	Abstract Types	Afferent Ca	Efferent Ce	Instability	Abstractness
Assembly-CSharp v0.0.0.0	10,422	64,806	243	9	7	336	0.98	0.04
Assembly-CSharp-Editor v0.0.0.0	192	905	8	0	0	56	1.00	0.00

Assembly-CSharp has **instability I = 0.98** — highly unstable, meaning its types are used by relatively few external assemblies but depend heavily on external types. Combined with abstractness of 0.04 (almost entirely concrete), this places it in the **"Zone of Pain"** on the Abstractness vs Instability diagram — concrete and stable, painful to modify. This is the expected position for a monolithic Unity game assembly but confirms that introducing interfaces and abstractions (per SOLID DIP) is the correct refactoring direction.

5.5 Overall Issue Summary (NDepend)

Total Issues	Blocker	Critical	High	Medium	Low
1,001	0	1	291	641	68

Technical debt: 31 days total. Debt rating: C. Effort to reach rating B: 5 days 2 hours. Debt as percentage of estimated development effort: 12% (within the 20% warning threshold). Maximum method cyclomatic complexity: 53. Average method complexity: 2.74.

6. CodeScene Analysis Results

CodeScene analysed the full iDaVIE git history (2018–May 2026) with latest results from 19 May 2026. The analysis covers 23,116 LOC across 620 files with 2 active developers in the most recent period.

6.1 Code Health Overview

Healthy (Green, score 9–10)	Problematic (Yellow, score 4–8)	Unhealthy (Red, score 1–3)
35% — 8,091 LOC	43% — 9,940 LOC	22% — 5,086 LOC
Hotspot Code Health	Average Code Health	Worst Performer
5.5 (Problematic) — stable over 1 year	7.0 (Problematic) — stable over 1 year	1.9 (Unhealthy) — slightly worse than 1 year ago (was 2.0)

Active risks: 10 files with declining Code Health trend. 13 possible ex-developers (original iDaVIE maintainers, last contributions 2018–2025 — not a current risk given 100% Code Familiarity score).

6.2 Hotspot Rankings

Files ranked by development activity (commit frequency × file size). Health scores on a 1–10 scale (1 = worst). Files with health ≤3 are Unhealthy; 4–8 Problematic; 9–10 Healthy.

File	LOC	Commits	Health Score	Trend	Status	CodeScene Issues
VolumeDataSet.cs	1,597	3	1.92	↓ Declining	UNHEALTHY	File Size · Brain Method · Bumpy Road · Deep Nesting
data_analysis_tool.cpp	581	2	2.10	→ Stable	UNHEALTHY	Brain Method · Bumpy Road · Deep Nesting · Many Conditionals
CanvassDesktop.cs	1,540	4	3.53	→ Stable	UNHEALTHY	Low Cohesion · File Size · Bumpy Road · Deep Nesting
DesktopPaintController.cs	1,207	3	3.65	↓ Declining	UNHEALTHY	File Size · Bumpy Road · Deep Nesting · Many Conditionals
VolumeInputController.cs	1,225	4	5.78	↓ Declining	PROBLEMATIC	Low Cohesion · File Size · Bumpy Road · Deep Nesting
FeatureSetRenderer.cs	532	2	5.83	→ Stable	PROBLEMATIC	Bumpy Road · Deep Nesting · Many Conditionals · Complex Method

VolumeDataSetRender.cs	1,127	4	5.90	→ Stable	PROBLEMATIC	Low Cohesion · Bumpy Road · Complex Method · Complex Conditional
fits_reader.cpp	793	2	5.95	→ Stable	PROBLEMATIC	Bumpy Road · Deep Nesting · Complex Method · Complex Conditional
VideoScriptReader.cs	192	3	6.12	↓ Declining	PROBLEMATIC	Complex Method · Complex Conditional · Large Method
VolumeCommandController.cs	610	2	7.67	→ Stable	PROBLEMATIC	Complex Method · Duplicated Blocks · Large Method
FitsReader.cs	616	2	7.79	→ Stable	PROBLEMATIC	Many Conditionals · Complex Method · High Code Duplication · Excess Params
ShapesManager.cs	355	2	8.09	↓ Declining	HEALTHY	Bumpy Road · Deep Nesting · Complex Method
FeatureSetManager.cs	368	2	8.22	→ Stable	HEALTHY	Deep Nesting · Complex Method · Complex Conditional · Excess Params
VideoCameraController.cs	416	2	9.24	→ Stable	HEALTHY	Bumpy Road · Complex Method · Complex Conditional

DesktopPaintController.cs (health 3.65, declining into Risky) was not included in the original Understand CK scan. It has 1,207 LOC and 3 commits. ST6 (Apaties III) owns this file alongside CanvassDesktop.cs — two critically unhealthy files in one sub-team represents a significant workload risk.

6.3 Temporal Coupling (Change Coupling)

Files that change together in the same commit more often than expected by chance, indicating hidden dependencies not visible in the static dependency graph. Minimum coupling threshold shown: 20%.

File A	File B	Coupling	Revisions
DataAnalysis.cs	data_analysis_tool.cpp	67%	27
AstTool.cs	ast_tool.cpp	66%	9
FeatureSetManager.cs	FeatureSetRenderer.cs	60%	38
Feature.cs	FeatureSetRenderer.cs	56%	30
Feature.cs	FeatureSetManager.cs	54%	31
FitsReader.cs	fits_reader.cpp	51%	22
VolumeDataSet.cs	VolumeDataSetRenderer.cs	48%	103
CatalogDataSetRenderer.cs	DataMapping.cs	45%	11
MomentMapMenuController.cs	MomentMapRenderer.cs	45%	20
VolumeCommandController.cs	VolumeInputController.cs	38%	67
VolumeDataSetRenderer.cs	VolumeInputController.cs	38%	97
CanvassDesktop.cs	VolumeDataSetRenderer.cs	21%	95

Strongest finding: VolumeDataSet.cs ↔ VolumeDataSetRenderer.cs: 48% coupling over 103 revisions. Three independent tools (SonarQube, Understand CK, CodeScene) all flag these files — temporal coupling confirms the architectural boundary between them is already broken in practice and must be the primary refactoring target.

New hidden dependency: FeatureSetManager.cs ↔ FeatureSetRenderer.cs: 60% coupling over 38 revisions. These appear to be separate classes in the static dependency graph but change together more often than not. An interface boundary should be introduced before Sprint 2 refactoring begins.

6.4 Knowledge Distribution

Code Familiarity	Knowledge Islands
100% — Good. All code is written by currently active developers. No orphaned code from ex-developers.	15% — Good. 15% of code has key personnel dependencies (single-author files). Operational risk increases if those files also have low health.

The 13 possible ex-developers identified by CodeScene are the original iDaVIE maintainers (Cilliers Pretorius, Fabio Vitello, Angus Comrie, TommVC, and others — last contributions between 2018 and 2025). The 100% Code Familiarity score confirms Team Beta’s fork has sufficient coverage of these files. **The knowledge risk map identifies the UI directory as the primary knowledge island area**, consistent with CanvassDesktop.cs being one of the most complex files in the codebase.

Coordination Needs: The VolumeData directory shows the highest coordination need (rated High by CodeScene), followed by the Menu directory. This is consistent with the temporal coupling data and the NDepend namespace cycle findings — VolumeData is the most architecturally entangled area of the codebase.

6.5 Churn Analysis

The bulk of codebase churn occurred during active iDaVIE development in 2019–2021 (peaks of ~35,000 raw churn lines). Since 2022 the codebase has been in low-activity maintenance mode with 1–2 active contributors. Active contributor count peaked at 7 (circa 2020) and has been 1–2 since 2022. **The god-classes grew during the 2019–2021 burst and have been largely untouched since** — they accumulated complexity rapidly and then calcified. This explains both their high WMC and the relatively low commit counts on the hotspot list: these files were built fast, left complex, and rarely changed despite being the most important files in the codebase.

The 2025–2026 churn spike represents Team Beta's fork activity. The fact that VolumeDataSet.cs and CanvassDesktop.cs are already showing declining health in this short window confirms that even modest changes to these god-classes are causing measurable quality deterioration.

7. Interpretation Against ISO/IEC 25010 Sub-characteristics

Assessment now draws on all four tool sets. Column headers indicate which tool provides evidence for each sub-characteristic rating.

Sub-characteristic	SonarQube evidence	CK evidence (Understand)	NDepend evidence	CodeScene evidence	Risk
Modularity	Menu/ 14.4% dup; fits_reader.cpp 13.9% dup; PluginInterface 7.7% dup.	CBO: CanvassDesktop 47, VolumeDataSetRenderer 44, VolumeInputController 40 — all breach ≤25 limit.	Namespace cycles confirmed (ND1401). Mutual dependency CatalogData ↔ VolumeData (119 members, ND1400).	VolumeDataSet ↔ VolumeDataSetRenderer 48% coupling, 103 joint revisions. VolumeData highest coordination need.	CRITICAL
Analysability	783 C# + 315 C++ maintainability issues. CanvassDesktop 11 reliability issues. AstTool.cs 70 issues / 47 LOC.	WMC: VolumeDataSet 278, VolumeInputController 260, CanvassDesktop 247. MaxCyclomatic 49 on VolumeCommandController.	1,001 total issues. Max cyclomatic complexity 53. Technical debt 31 days (rating C).	4 hotspot files score ≤3.65 (Unhealthy). 10 files with declining health trends.	HIGH
Modifiability	Menu/ dup 14.4%; PaintMenuController 20.4%; QuickMenuController 14.9% — both exceed 5% threshold significantly.	LCOM: 4 classes above 94%. High LCOM = high change risk per method modification.	8 critical rules violated. Non-readonly static fields risk state mutation across threads.	DesktopPaintController.cs declining into Risky. FeatureSetManager ↔ FeatureSetRenderer 60% coupling, 38 revisions.	HIGH
Testability	0.0% coverage. MonoBehaviour coupling throughout god-classes.	CBO 44 on VolumeDataSetRenderer means 44 mocked dependencies for a unit test — effectively untestable without Unity scene.	13 coverage-related quality gates skipped (no test data). VideoCameraController.SaveFrames() creates raw threads.	Knowledge Islands: 15% of code. Code Familiarity: 100% (positive — no orphaned code from ex-developers).	HIGH
Reusability	PluginInterface wrappers exist but ABI not versioned. C++ duplication between layers.	LCOM >80% on all major classes — low cohesion means extraction is hard. FeatureTable (LCOM 33%) is the most reusable candidate.	Instability I=0.98 for Assembly-CSharp — highly unstable, concrete, and painful to modify.	VideoMaker directory: multiple files with acceptable health (≥9.24) indicating some reusable components exist.	MEDIUM-HIGH

8. Remaining Coverage Gaps

The following gaps remain open. NDepend, CodeScene, and DV8 (substituted by NDepend DSM — see rationale in gap row below) are now closed.

Gap	Owner	Due
DV8 DSM — CLOSED via substitution: DV8 could not produce a valid DSM output for Assembly-CSharp.dll (Unity scripting assembly format incompatibility). DSM requirement satisfied by NDepend dependency matrix and namespace cycle analysis (ND1400/ND1401). Spec Section 7.2 cites 'DV8 or NDepend' for DSM — NDepend fulfils this.	Architecture Guild + Quality Guild	CLOSED — substituted
HLSL shader files (non-C#): manual complexity review	ST3 QC	Sprint 2
Understand CK for native C++ (ast_tool.cpp, fits_reader.cpp, etc.)	ST2 QC	CLOSED — not applicable: native C++ files in native_plugins_cmake/ are procedural (no class definitions), making CK metrics inapplicable. Coverage provided by SonarQube (315 maintainability issues, 2,308 LOC) and CodeScene (data_analysis_tool.cpp health 2.10, fits_reader.cpp health 5.95).

9. Next Steps and Priorities

Priority	Action	Owner	Due
P1	Triage CanvassDesktop.cs 11 reliability issues and FeatureMenuCell.cs 5 reliability issues	ST6 + ST5 QC	Day 3
P1	Resolve namespace dependency cycle (ND1401): VolumeData ↔ UI ↔ VoTableReader — 7-namespace cycle, 2h estimated debt	ST1 / Architecture Guild	Sprint 2 Day 1
P1	Resolve mutually dependent namespaces (ND1400): CatalogData ↔ VolumeData — 119 members, must reach 0 cycles	ST1 / Architecture Guild	Sprint 2 Day 1
P1	Triage VideoCameraController critical issue and raw thread creation (ND2500) — must not reach production	VideoMaker owner	Day 3
P2	Investigate FeatureSetManager / FeatureSetRenderer 60% temporal coupling — plan interface boundary	ST5 QC	Sprint 2 Day 1
P2	Investigate PaintMenuController / QuickMenuController duplication — plan extraction	ST4 QC	Sprint 2 Day 1
P2	Begin FitsReader.cs SRP decomposition per ST2 refactoring plan	ST2	Sprint 2
P2	Address DesktopPaintController.cs — newly identified declining hotspot (3.65, not in CK baseline)	ST6	Sprint 2
P3	Produce Day 13 projected CK deltas per worked refactoring example	All sub-team QCs	Day 13 (Tue 3 Jun)

Data sources: SonarQube Cloud (EPIC-Team-Beta/iDaVIE, main, 20 May 2026); Understand CSV export (20 May 2026); NDepend v2026.1.5 (20 May 2026); CodeScene (19 May 2026). Deliverable T2 under CS4443